

Università degli Studi di Salerno

Corso di Ingegneria del Software

Easy Work Management System

Test Plan

Versione 1.0



E.W.M.S.

Easy Work Management System

Data: 06/02/2026

Progetto: Easy Work Management System	Versione: 1.0
Documento: Test plan	Data: 06/02/2026

Coordinatore del progetto:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Partecipanti:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Scritto da:	Vincenzo Mauro, Giovanni Robustelli
--------------------	-------------------------------------

Revision History

Data	Versione	Descrizione	Autore
19-12-2025	0.1	Strutturazione documento e inserimento punto 9	Giovanni Robustelli
20-12-2025	0.5	Sono stati aggiunti tutti i punti previsti, si prevede l'aggiunta di nuovi test cases	Giovanni Robustelli
06/02/2026	1.0	Inserimento specifiche per unit testing e integration testing	Giovanni Robustelli

Indice

1.	Introduzione	4
2.	Relazione con gli altri documenti	4
3.	System overview	4
4.	Funzionalità da testare/da non testare	5
5.	Pass/Fail criteria	6
6.	Approccio	7
7.	Sospensione e ripristino	8
8.	Materiale per i test	9
9.	Test cases	11
9.1	Unit Testing	11
9.2	Integration testing	11
9.2.1	PersistenceManagemet	11
9.2.2	Integration AccountManagement + PersistenceManagement	12
9.2.3	Integration AccessManagement + PersistenceManagement	14
9.2.4	Integration TaskManagement + PersistenceManagement	15
9.3	System testing	18
9.3.1	Test_Login (TC_1)	18
9.3.2	Test_Creazione_Task (TC_2)	19
9.3.3	Test_Inserimento_account (TC_3)	21
10	. Testing Schedule	23

1. Introduzione

Questo documento ha l'obiettivo di definire la strategia di testing e le specifiche di validazione per la piattaforma Easy Work Management System (E.W.M.S.). In questo documento saranno presentate le strategie per il testing per il sistema. Saranno esplorate le tecniche che il team applicherà per effettuare unit testing, integration testing e system testing. In particolare, saranno elencati i test cases per le funzionalità critiche del sistema.

2. Relazione con gli altri documenti

Per la stesura di questo documento si è fatto riferimento ai seguenti documenti:

- **Requirement Analysis Document:** tale relazione si fonda sul fatto che i test case critici scelti si sono basati sui requisiti funzionale e non funzionali descritti all'interno del documento con particolare attenzione anche ai casi d'uso che prevedevano una forte interazione da parte degli attori del sistema.
- **System Design Document:** tale relazione si fonda sulla suddivisione in sottosistemi del sistema EWMS, grazie a questo documento è stato possibile individuare correttamente i sottosistemi che partecipano ai test case descritti in questo documento.
- **Object Design Document:** tale relazione è giustificata dal fatto che il documento contiene la descrizione delle classi che entreranno in gioco nei testing.

3. System overview

L'applicazione E.W.M.S. è una web application basata su una three-tier architecture, come descritto nel System Design Document. Questa struttura consente una netta separazione delle responsabilità, facilitando la decomposizione gerarchica necessaria per il testing modulare:

- **Presentation Layer:** Gestisce l'interazione con l'utente finale tramite interfacce web dinamiche. È il livello in cui vengono acquisiti i dati di input attraverso i form, visualizzati i feedback del sistema e gestito il flusso di controllo.
- **Application Layer:** Rappresenta il nucleo dell'applicazione dove risiede la business logic. In questo livello, i vari service elaborano le richieste e applicano i vincoli definiti nel RAD.

- Data Layer: Responsabile della persistence dei dati, interagisce con il database relazionale. Utilizza il pattern DAO (Data Access Object) per garantire l'integrità delle informazioni durante i processi di lettura e scrittura.

Il sistema punta a centralizzare l'assegnazione dei compiti e il monitoraggio dello stato delle attività, riducendo i tempi di coordinamento e aumentando l'efficienza e la trasparenza organizzativa.

4. Funzionalità da testare/da non testare

In questa sezione viene riportata l'applicazione sistematica del metodo category partition per le principali funzionalità del sistema E.W.M.S., identificate come critiche nel Requirement Analysis Document. L'obiettivo è trasformare i functional requirements in una serie di test frames che garantiscano una copertura esaustiva degli scenari di utilizzo, dei casi di error e dei boundary values. Le funzionalità che sono state scelte per avere un testing più approfondito sono:

- Login
- Creazione di un nuovo task
- Creazione di un nuovo account da aggiungere al sistema

Il sistema presenta diverse altre funzionalità, le quali però hanno un'interazione leggera con gli attori del sistema o hanno un funzionamento molto simile alle funzionalità testate, la loro fase di testing quindi si condurrà principalmente all'interno di unit e integration testing.

5. Pass/Fail criteria

In questa sezione vengono definiti i criteri oggettivi per determinare l'esito delle attività di testing condotte sul sistema E.W.M.S.. La valutazione si basa sulla coerenza tra il comportamento osservato dell'applicazione e le specifiche tecniche formalizzate nel Requirement Analysis Document e nell'Object Design Document.

Criteri di Superamento

Un **test case** è considerato superato quando il sistema soddisfa contemporaneamente le seguenti condizioni:

- **Integrità della Logica:** L'output prodotto corrisponde esattamente ai risultati attesi definiti nell'oracolo del caso di test.
- **Consistenza della Persistenza:** Le operazioni di CRUD effettuate tramite il pattern DAO si riflettono correttamente nel database. (Ad esempio, la creazione di un dipendente deve generare portare anche alla generazione di un'entry all'interno della tabella Dipendente).
- **Feedback dell'Interfaccia:** La web interface mostra il messaggio di conferma o di errore previsto.

Criteri di Fallimento

Un **test case** è considerato fallito se si verifica anche solo una delle seguenti condizioni:

- **Deviazione dalle Specifiche:** Il sistema produce un output differente da quello previsto, o non attiva le post-condizioni definite nell'ODD.
- **Mancata Validazione:** Il sistema accetta dati di input che violano i vincoli di boundary values (es. accettazione di una password troppo corta o di una data di scadenza passata).
- **Inconsistenza dei Dati:** L'operazione non viene riportata correttamente nel persistence layer (es. dati mancanti o errati nel database nonostante un messaggio di successo).
- **Eccezioni non Gestite:** L'esecuzione del test solleva una exception non intercettata (es. NullPointerException o SQLException) che causa l'interruzione del processo o la visualizzazione di una pagina di errore generica non formattata.

Criteri per la qualità della fase di testing

La fase di testing è una delle parti fondamentali durante lo sviluppo di un sistema. Il suo svolgimento può anche coprire il 25% degli sforzi totali di sviluppo sia in senso di tempo che in senso di costo. Quindi la **qualità** della fase di testing viene misurata in base a quanti test hanno effettivamente provocato delle failure all'interno del sistema. Per un testing di successo si prevede che la maggior parte dei test case falliscano alla fine di individuare tutte i possibili difetti che inevitabilmente saranno presenti durante lo sviluppo del sistema.

6. Approccio

L'approccio adottato per la verifica del sistema E.W.M.S. segue una metodologia di black-box testing rigorosa integrata con fasi di white-box testing volte a dare copertura totale al testing delle singole parti del sistema.

Unit testing

L'obiettivo dello unit testing è verificare la correttezza logica dei singoli componenti del sistema in modo isolato. Per E.W.M.S., questo livello di testing si focalizza sulle classi definite nell'Object Design Document e si concentrerà sui servizi offerti dai sottosistemi del livello applicativo e di persistenza.

- **Analisi dei contratti:** Verifica rigorosa delle pre-conditions e post-conditions specificate nell'ODD.
- **Boundary Value Analysis:** Test dei limiti per ogni parametro di input (es. lunghezze stringhe, valori nulli).
- **Mocking:** Utilizzo di mock objects per isolare l'unità sotto esame dalle dipendenze esterne, come il database o il web container, garantendo che l'errore rilevato appartenga esclusivamente all'unità testata.

Per effettuare i test si procederà tramite category partition.

Integration testing

Per il sistema E.W.M.S., si è deciso di adottare la tecnica del **bottom-up testing**. Questo approccio prevede di iniziare il test dai componenti di livello più basso (indipendenti da altri moduli), per poi integrarli progressivamente nei moduli di livello superiore, eliminando la necessità di utilizzare stubs.

La decomposizione gerarchica del sistema per questa fase prevede la seguente progressione:

- **Livello Iniziale:** Costituito dal persistence layer (DAO), che viene testato per primo, eventualmente con l'ausilio di test drivers.
- **Livello Intermedio:** Rappresentato dai componenti dell'application layer (dove risiede la logica di business che vengono integrati e testati utilizzando i moduli del livello inferiore già verificati).
- **Livello Finale:** Composto dai controller del presentation layer, non verranno testati nella fase di integrazione, ma saranno testati durante la fase di system testing.

System testing

Il system testing rappresenta la fase finale di verifica, in cui l'intera applicazione viene testata come un'unica entità per accertarsi che soddisfi i functional requirements descritti nel Requirement Analysis Document.

L'Oggetti del test sarà l'intero processo software. La tecnica utilizzata sarà **category partition**: Applicazione sistematica delle tabelle di partizione generate (es. per il login o la creazione di un task) per coprire tutti gli scenari d'uso nominali e di error delle funzionalità critiche descritte all'interno di questo documento.

7. Sospensione e ripristino

In questa sezione vengono definiti i criteri secondo i quali le attività di testing sul sistema E.W.M.S. devono essere temporaneamente interrotte e le condizioni necessarie per la ripresa del processo di verifica. Questa gestione assicura che le risorse non vengano sprecate in presenza di bug bloccanti che impediscono la progressione dei test.

Criteri di Sospensione

L'attività di testing viene sospesa qualora si verifichi una delle seguenti condizioni critiche:

- **Bug bloccanti:** Presenza di malfunzionamenti critici nelle funzionalità core che impediscono l'accesso ad altre aree del sistema. (Ad esempio: impossibilità di inizializzare la sessione utente o errori fatali nel connection pool).
- **Elevato tasso di fallimento:** Se oltre il 40% dei test frames eseguiti in una specifica sessione produce un esito di fail, indicando un'instabilità diffusa della build corrente.
- **Corruzione dei dati:** Rilevamento di anomalie gravi nel persistence layer che compromettono l'integrità dei record durante i test di integrazione o di sistema.

7.2 Requisiti di Ripristino

Una volta sospesa l'attività, il **testing** può essere ripreso solo dopo il soddisfacimento dei seguenti requisiti:

- **Risoluzione delle anomalie:** Il team di sviluppo deve fornire una nuova build in cui il bug che ha causato la sospensione è stato corretto e verificato tramite uno unit testing preliminare.
- **Ripristino dell'ambiente:** Verifica della corretta configurazione del server di deployment e ripristino della consistenza del database (attraverso procedure di backup e recovery se necessario).
- **Disponibilità della documentazione:** Aggiornamento dell'Object Design Document o del System Design Document qualora la correzione del problema abbia comportato modifiche strutturali alle class interfaces o alla logica di service.

L'obiettivo di questa strategia è garantire che il modello del sistema mantenga standard qualitativi elevati, evitando che errori strutturali nello strato della business logic o incongruenze con il modello si propaghino nelle fasi avanzate di validazione.

8. Materiale per i test

Per l'esecuzione dei test sul sistema E.W.M.S., verranno adottati strumenti specifici del software engineering per l'ambiente Java, garantendo l'automazione dei processi e la ripetibilità dei test cases. La scelta degli strumenti è dettata dalla necessità di coprire i tre livelli di testing definiti nella metodologia, supportando la three-tier architecture del progetto.

JUnit

Rappresenta il framework di riferimento per il testing in linguaggio Java. Consente di definire ed eseguire test automatizzati attraverso l'uso di annotazioni e asserzioni, facilitando la verifica puntuale del comportamento delle singole unità di codice e l'organizzazione dei test in Suites riutilizzabili.

Selenium IDE

È uno strumento di Record and Playback utilizzato principalmente per il System Testing e la verifica delle interfacce web. Permette di registrare le interazioni dell'utente con le applicazioni e riprodurle automaticamente, assicurando che i flussi funzionali completi siano rispettati e che non vi siano regressioni visibili lato Frontend.

Supporto allo Unit e Integration Testing

L'architettura dei test, specialmente per le fasi di Unit Testing e Integration Testing, si avvale di strumenti specifici per la gestione delle dipendenze e della persistenza:

- **Mockito:** Questa libreria di Mocking viene utilizzata per isolare le unità di codice durante lo Unit Testing nello specifico dell'application layer del sistema EWMS. Permette di creare oggetti Mock che sostituiscono le dipendenze reali, consentendo di testare la logica di business in un ambiente controllato e deterministico, senza la necessità di componenti esterni attivi.
- **H2 Database:** Per i test di unit testing del livello di persistenza e per la fase di integration testing. È un In-memory Database relazionale che permette di eseguire i test contro un database reale residente nella memoria volatile. Questo approccio garantisce che i test siano estremamente veloci e che ogni esecuzione parta da uno stato pulito, permettendo di validare correttamente le query SQL, i vincoli di integrità e il mapping dei dati senza sporcare il database di produzione.

Strumento per il System Testing: Selenium IDE

Per la validazione end-to-end dell'applicazione, verrà utilizzato Selenium IDE.

- **Motivazione:** Permette di automatizzare le azioni degli utenti sui browser web. È ideale per verificare i flussi descritti nel RAD, interagendo direttamente con il presentation layer.
- **Utilizzo:** Verrà impiegato per simulare scenari completi, come il login, la creazione di un task (compreso l'upload di allegati). Selenium controllerà che, a fronte di input validi o errati, il DOM della pagina mostri correttamente i messaggi di successo o i messaggi di error definiti nei requisiti.

Ambiente di Esecuzione

I test di unità e integrazione saranno effettuati all'interno dell'IDE, per i test di sistema invece verrà utilizzato Selenium IDE, l'esecuzione del server sarà da ambiente di produzione.

9. Test cases

9.1 Unit Testing

Per leggibilità del documento il team ha deciso di inserire i **test cases** e **test cases specification** per le classi sviluppate all'interno di documenti separati raggruppandoli seguendo i sottosistemi descritti all'interno dell'SDD nella forma: `UnitTest_TestCases_[NomeSottosistema]_EWMS` e `UnitTest_CasesSpecification_[NomeSottosistema]_EWMS`.

9.2 Integration testing

Durante l'integration testing per il sistema EWMS, prima di ogni test case, il database viene completamente ripulito e lo schema viene ricreato da zero eseguendo lo script SQL di produzione. Questo garantisce l'indipendenza dei test, questo non rallenterà l'esecuzione di test grazie all'utilizzo dell'**in-memory database H2**. Le asserzioni non si limitano al valore di ritorno dei metodi del Service, ma verificano il Side-Effect sul database (es. presenza effettiva del record, rispetto dei vincoli UNIQUE, cancellazione a cascata) interrogando direttamente il DAO o utilizzando query SQL native.

9.2.1 PersistenceManagemet

Per effettuare testing bottom-up partiamo dal livello più basso. Dato che l'unit-test delle classi di questo sottosistema è stato effettuato utilizzando il database H2, invece che utilizzando Mockito, sappiamo già come si comporta in un sistema reale quindi non ci sarà bisogno testare questo sottosistema durante l'integration testing avendo già affrontato tutte le possibili incongruenze con il database durante appunto la fase di test precedente.

9.2.2 Integration AccountManagement + PersistenceManagement

Di seguito sono riportati i casi di test per integrare il sottosistema di AccountManagement.

ID Test Case	Descrizione	Pre-Condizioni	Dati di Input	Azioni	Oracolo
TC-INT-ACC-01	Aggiunta Account Multipli Verifica che il servizio persista correttamente diverse tipologie di utenti.	Database inizializzato e vuoto.	1. Utente Gestore 2. Utente Supervisore Password: "Goodpwd12!"	Invocazione di addAccount per entrambi gli utenti.	Nessuna eccezione lanciata. Il database contiene esattamente 2 record nella tabella utenti.
TC-INT-ACC-02	Recupero Dettagli Account Verifica il corretto mapping dei campi dal DB all'oggetto Java.	Un utente è già stato inserito nel DB tramite DAO.	Matricola dell'utente inserito.	Invocazione di getAccount(matricola).	L'oggetto restituito non è null. Tutti i campi (Nome, Cognome, Email, DataNascita) corrispondono ai dati inseriti.
TC-INT-ACC-03	Lista Utenti (DB Vuoto) Verifica il comportamento in assenza di dati.	Database vuoto.	Nessuno.	Invocazione di getAllAccount().	Viene restituita una lista vuota (size = 0).
TC-INT-ACC-04	Lista Utenti (Popolato) Verifica il recupero di tutti gli account presenti.	Database popolato con: 1 Gestore, 1 Supervisore, 1 Dipendente.	Nessuno.	Invocazione di getAllAccount().	Viene restituita una lista contenente esattamente 3 utenti .

TC-INT-ACC-05	Violazione Vincolo Email Verifica che il DB impedisca duplicati e il Service gestisca l'errore.	Un utente con email mario.rossi@... è già presente nel DB.	Nuovo utente con la stessa email mario.rossi@....	Invocazione di addAccount.	Viene lanciata l'eccezione EmailGiaPresenteException . Il record duplicato non viene inserito.
TC-INT-ACC-06	Filtro Supervisor Verifica che il metodo restituisca solo gli utenti con ruolo Supervisore.	Database popolato con: 1 Gestore, 1 Supervisore, 1 Dipendente.	Nessuno.	Invocazione di getAllSupervisor().	La lista risultante ha size = 1 . L'elemento contenuto corrisponde all'utente Supervisore (verifica nome/cognome).
TC-INT-ACC-07	Cancellazione e Integrità Referenziale Verifica la rimozione a cascata (Cascade Delete).	Un Supervisore è presente nel DB (record in tab. Utente e tab. Supervisore).	Oggetto Utente da cancellare.	Invocazione di deleteAccount.	1. findByMatricola ritorna null. 2. Query diretta SQL SELECT count(*) sulla tabella supervisore restituisce 0 (record eliminato fisicamente).

9.2.3 Integration AccessManagement + PersistenceManagement

ID Test Case	Descrizione	Pre-Condizioni	Dati di Input	Passi	Oracolo
TC-INT-ACS-01	Login Successo (Gestore) Verifica l'autenticazione corretta per un utente con ruolo Gestore.	Utente "Mario Rossi" (gestore) presente nel DB. Password salvata con Hash BCrypt.	Email: mario.rossi@azienda.it Password: goodpwd.1	Invocazione di login().	Il metodo restituisce true .
TC-INT-ACS-02	Login Successo (Dipendente) Verifica l'autenticazione per un ruolo subordinato (Dipendente).	Dipendente "Luigi Rossi" presente nel DB e associato a un Supervisore. Password hashata.	Email: luigi.rossi@azienda.it Password: goodpwd.1	Invocazione di login().	Il metodo restituisce true . (Conferma che la query attraversa correttamente le tabelle specifiche dei ruoli).
TC-INT-ACS-03	Login Fallito (Password Errata) Verifica il rifiuto dell'accesso in caso di mismatch della password.	Utente "Mario Rossi" presente nel DB.	Email: mario.rossi@azienda.it Password: PASSWORD_SBagliata_123	Invocazione di login().	Il metodo restituisce false .
TC-INT-ACS-04	Login Fallito (Utente Inesistente) Verifica il comportamento con email non presente nel sistema.	Database popolato, ma l'email di test non esiste.	Email: email.inesistente@ghost.com Password: qualasiasiPwd	Invocazione di login().	Il metodo restituisce false .
TC-INT-ACS-05	Recupero Dati Utente	Supervisore presente nel DB.	Email: supervisore.rossi@azienda.it	Invocazione di getUtente().	L'oggetto restituito non è null . I dati (Nome, Ruolo)

	Verifica il recupero dell'oggetto Utente post-autenticazione (utile per la sessione).				corrispondono a quelli nel DB.
TC-INT-ACS-06	Recupero Utente Inesistente Verifica la gestione di query vuote.	L'email di test non è presente nel DB.	Email: non.esisto@nulla.it	Invocazione di getUtente().	Il metodo restituisce null .

9.2.4 Integration TaskManagement + PersistenceManagement

TaskCommonService

ID Test Case	Descrizione	PreCondizioni	Dati di Input	Passi	Oracolo
TC-INT-TSK-01	Recupero Dettaglio Task Verifica lettura dati e associazioni FK.	Task #1 presente nel DB, associato a Sup. Rossi e Dip. Luigi.	ID del Task #1.	Invocazione di getTask(id).	Oggetto restituito non null. Tutti i campi corrispondono ai dati inseriti.
TC-INT-TSK-02	Hold Task (Stato Invalido) Verifica blocco sospensione su task non avviati.	Task #1 in stato DA_COMPLETARE.	ID del Task #1.	Invocazione di holdTask(id).	Viene lanciata IllegalStateException. Lo stato nel DB non cambia.
TC-INT-TSK-03	Hold Task (Successo) Verifica sospensione di un task in esecuzione.	Task #1 portato manualmente in stato IN_ESECUZIONE.	ID del Task #1.	Invocazione di holdTask(id).	Il metodo restituisce true. Lo stato nel DB diventa IN_SOSPENSIONE.
TC-INT-TSK-04	Hold Task (ID Inesistente)	Nessuna.	ID: 999999.	Invocazione di holdTask(id).	Il metodo restituisce false.

	Verifica gestione ID non validi.				
--	----------------------------------	--	--	--	--

TaskSupervisoreService

ID Test Case	Descrizione	PreCondizioni	Dati di Input	Passi	Oracolo
TC-INT-TSK-05	Creazione Task (Successo) Verifica inserimento e link FK.	Supervisore e Dipendente esistenti nel DB.	Titolo: "Nuovo Task" Istruzioni > 10 chars Date valide.	Invocazione di createTask(...).	Il task viene salvato nel DB. Le chiavi esterne (Dipendente/Supervisore) sono mappate correttamente.
TC-INT-TSK-06	Creazione Task (Errore FK) Verifica integrità referenziale su dipendente inesistente.	Supervisore esistente.	Matricola Dipendente: 99999 (Inesistente).	Invocazione di createTask(...).	Viene lanciata IllegalArgumentException (o eccezione di violazione vincolo DB).
TC-INT-TSK-07	Lista Task Supervisore Verifica filtro per specifici supervisori.	DB con 3 task: 2 del Sup. Base, 1 di un altro Sup.	Oggetto Supervisore Base.	Invocazione di getAllTaskSup(...).	La lista contiene esattamente 2 task . Il task dell'altro supervisore è escluso.
TC-INT-TSK-08	Info Dipendenti Assegnati Verifica recupero info dipendenti.	Dipendente Luigi assegnato a Sup. Rossi.	Matricola Sup. Rossi.	Invocazione di getAllDipendentiInfo.	La lista non è vuota. Contiene Nome/Cognome di "Luigi Rossi".
TC-INT-TSK-09	Cancellazione Task Verifica rimozione fisica.	Task esistente nel DB.	ID del task.	Invocazione di deleteTask(id).	Il metodo restituisce true. Una successiva ricerca findById restituisce null.

TaskDipendenteService

ID Test Case	Descrizione	PreCondizioni	Dati di Input	Azioni (Passi)	Risultato Atteso (Oracolo)
TC-INT-TSK-10	Lista Task Dipendente Verifica recupero task assegnati.	2 Task assegnati al Dipendente Luigi nel DB.	Oggetto Dipendente: Luigi.	Invocazione di getAllTaskDip.	La lista contiene entrambi i task previsti.
TC-INT-TSK-11	Inizializzazione Task (Successo) Transizione Start Lavori.	Task #1 in stato DA_COMPLETARE.	ID Task #1.	Invocazione di inizializzaTask(id).	Restituisce true. Stato nel DB aggiornato a IN_ESECUZIONE.
TC-INT-TSK-12	Inizializzazione Task (Fail) Violazione pre-condizione su task finito.	Task #1 portato in stato COMPLETATO.	ID Task #1.	Invocazione di inizializzaTask(id).	Viene lanciata IllegalStateException.
TC-INT-TSK-13	Completamento Task (Successo) Transizione Fine Lavori.	Task #1 portato in stato IN_ESECUZIONE.	ID Task #1.	Invocazione di completeTask(id).	Restituisce true. Stato nel DB aggiornato a COMPLETATO.
TC-INT-TSK-14	Completamento Task (Fail) Violazione pre-condizione ordine lavori.	Task #1 in stato DA_COMPLETARE.	ID Task #1.	Invocazione di completeTask(id).	Viene lanciata IllegalStateException.
TC-INT-TSK-15	Operazioni ID Inesistente Robustezza servizi dipendente.	Nessuna.	ID: 99999.	Invocazione di inizializzaTask e completeTask.	Entrambi i metodi restituiscono false.

9.3 System testing

In questa sezione verranno inserite le tabelle del Category Partition per le funzionalità selezionate per il testing, ogni test case avrà un Test case id (TC_id) e un nome utilizzati per il riferimento nel documento di Test Case Specification.

9.3.1 Test_Login (TC_1)

Category	Partizione	Vincoli
Username (email)	Valido (presente nel db)	[property Valido]
	Inesistente	[error]
	Formato non valido	[error]
	Null o vuoto	[error]
Password	Corretta (corrispondente all'username)	[if Valido]
	Errata	[if Valido] [error]
	Null o vuota	[error]

Tabella formati

Campo	Formato
Username (email)	^[a-zA-Z0-9]{1,50}\.[a-zA-Z0-9]{1,50}@azienda\.it\$
Password	^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@\$!%*?&])[A-Za-z\d@\$!%*?&]{8,16}\$

Test frames

ID Frame	Username	Password	Risultato Atteso (Output)
TF_1.1	Valido	Corretta	Login Successo: Redirect a Homepage
TF_1.2	Valido	Errata	Error: "Username o password non corrette"
TF_1.3	Inesistente	Qualsiasi	Error: "Username o password non corrette"
TF_1.4	Null	Qualsiasi	Error: "Il campo e-mail è obbligatorio"
TF_1.5	Valido	Null	Error: "Il campo password è obbligatorio"
TF_1.6	Formato Errato	Qualsiasi	Error: "Il formato e-mail non valido"

9.3.2 Test_Creazione_Task (TC_2)

Category	Partizione	Vincoli
Titolo	Valido	[property TitoloOK]
	Vuoto	[error]
	Formato non valido	[error]
Dipendente	Selezionato (presente in lista)	[property DipSelected]
	Non selezionato	[error]
Data di scadenza	Futura	[property DataOK]
	Passata	[error]
	Vuota	[error]
Priorità	Selezionata	[property PrioritàOK]
	Non selezionata	[error]
Istruzioni	Valida	[property IstruzioniOK]
	Formato non valido	[error]
Allegato	Valido	[property FileOK]
	Formato non supportato	[error]

	Dimensione eccessiva	[error]
	Assente	

Tabella formati

Campo	Formato
Titolo	^[a-zA-Z0-9\s.,\-_!]{1,50}\$
Data di scadenza	^d{4}-(0[1-9] 1[0-2])-(0[1-9] [12][0-9] 3[01])\$ (formato YYYY-MM-DD)
Istruzioni	^[s\S]{10,2000}\$
Allegato	.pdf, .docx, .xlsx, dimensione < 10 MB

Test frames

ID Frame	Titolo	Dipendente	Data	Istruzioni	Allegato	Risultato Atteso (Output)
TF_2.1	Valido	Selezionato	Futura	Valida	Assente	Success: Task creato, Redirect a Homepage
TF_2.2	Vuoto	Selezionato	Futura	Valida	N/A	Error: "Il campo 'Titolo' è obbligatorio"
TF_2.3	Valido	Non selezionato	Futura	Valida	N/A	Error: "Per favore scegliere un dipendente"
TF_2.4	Valido	Selezionato	Errata	Valida	N/A	Error: "Data di scadenza obbligatoria e in formato valido"
TF_2.5	Valido	Selezionato	Futura	Formato non valido	N/A	Error: "Inserire istruzioni (min. 10, max 2000 caratteri)"

9.3.3 Test_Inserimento_account (TC_3)

Category	Partizione	Vincoli
Nome	Valido	[property NomeOK]
	Vuoto	[error]
	Formato non valido	[error]
Cognome	Valido	[property CognomeOK]
	Vuoto	[error]
	Formato non valido	[error]
Email	Valido	[property EmailOK]
	Formato non valido	[error]
	Già presente nel DB	[error]
	Vuoto	[error]
Data di nascita	Valido	[property DataOK]
	Formato non valido	[error]
	Vuoto	[error]
Ruolo	Selezionato	
	Non selezionato	[error]
Matricola	Generata dal sistema	[property MatGenOK]
Password	Generata	[property passgenOK]
	Non generata	[error]

Tabella formati

Campo	Formato
Nome	^[a-zA-Z]{1,50}\$
Cognome	^[a-zA-Z]{1,50}\$

Email	^[a-zA-Z0-9]{1,50}\.[a-zA-Z0-9]{1,50}@azienda\.it\$
Data di nascita	^\d{4}-(0[1-9] 1[0-2])-(0[1-9] [12][0-9] 3[01])\$ (formato YYYY-MM-DD)
Password	^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@\$!%*?&])[A-Za-z\d@\$!%*?&]{8,16}\$

Test frames

ID Frame	Nome	Cognome	Data di Nascita	E-mail	Ruolo	Password	Risultato Atteso (Output)
TF_3.1	Valido	Valido	Valido	Valida (nuova)	SUPERVISORE	Generata	Success: Account creato e salvato
TF_3.2	Valido	Valido	Valido	Già esistente	SUPERVISORE	Generata	Error: "Indirizzo e-mail già utilizzato"
TF_3.3	Vuoto	Valido	Valido	Valida	SUPERVISORE	Generata	Error: "Il campo 'nome' è obbligatorio"
TF_3.4	Valido	Valido	Formato non valido	Valida	SUPERVISORE	Generata	Error: "Formato data non valido"
TF_3.5	Valido	Valido	Vuoto	Valida	SUPERVISORE	Generata	Error: "Il campo 'data di nascita' è obbligatorio"
TF_3.6	Valido	Valido	Valido	Formato errato	SUPERVISORE	Generata	Error: "Il formato e-mail non è valido"
TF_3.7	Valido	Valido	Valido	Valida	SUPERVISORE	Vuota	Error: "Password obbligatoria"
TF_3.8	Formato non valido	Valido	Valido	Valida	SUPERVISORE	Generata	Error: "Nome troppo lungo (Max 50 caratteri)"

TF_3.9	Valido	Vuoto	Valido	Valida	SUPERVISORE	Generata	Error: “Campo ‘cognome’ obbligatorio”
TF_3.10	Valido	Formato non valido	Valido	Valida	SUPERVISORE	Generata	Error: “Il cognome è troppo lungo (max 50).”
TF_3.11	Valido	Valido	Valido	Valida	Dipendente(Supervisore non selezionato)	Generata	Error: “Seleziona un supervisore.”

10. Testing Schedule

Le attività di testing per il sistema E.W.M.S. non sono concepite come una fase isolata al termine dello sviluppo, ma sono integrate nell'intero ciclo di vita del software engineering. Il processo prevede che lo unit testing venga eseguito non appena le singole unità logiche risultano pronte e compilabili. Questo approccio incrementale consente l'individuazione immediata di eventuali regressioni o difetti strutturali, riducendo i costi di correzione nelle fasi avanzate di integrazione.

Inoltre, la suite di test è considerata un'entità dinamica. Durante lo sviluppo, il team aggiungerà nuovi test cases man mano che lo sviluppo porta ad un miglior comprensione del sistema e dei suoi requisiti o qualora emergano scenari d'uso non previsti inizialmente o nel caso in cui la scomposizione gerarchica delle nuove funzionalità riveli ulteriori boundary values critici. Questo garantisce che la copertura dei functional requirements rimanga completa e aggiornata rispetto all'evoluzione della business logic e delle interfacce definite nell'Object Design Document.